

The White Dwarf Mass–Radius Relation: A Numerical Study Using the Chandrasekhar Equation of State

Bhaskar Jain, Anika Chawla, Ellen Wang, Amita Anand

PHYS 6260: Computational Astrophysics

Spring 2026

1 Introduction

White dwarfs are the end state for roughly 97% of all stars in the Galaxy, including our own Sun for the Earth [1]. When a low or intermediate mass star ($M \lesssim 8 M_{\odot}$) runs out of nuclear fuel, it sheds its outer layers as a planetary nebula, which is an interesting and cool process, and leaves behind a degenerate carbon-oxygen core, typically containing $0.5\text{--}1.2 M_{\odot}$ of material squeezed into a volume roughly the size of Earth (quite dense). With no nuclear burning capability left because of the lack of fuel, the whole thing is held up by the quantum mechanical degeneracy pressure of the electron gas. This is a phenomenon first explained by Fowler [2].

This entire deal was put in a firm theoretical footing by Chandrasekhar, who in 1931 showed that a cold degenerate star cannot exceed a certain maximum mass. Now we call this the Chandrasekhar limit [3][4]. The key insight is that as the stellar mass grows, the electrons get more and more relativistic, and in the fully relativistic limit the equation of state softens from $\Gamma = 5/3$ toward $\Gamma = 4/3$, at which point hydrostatic equilibrium can no longer be maintained.

We will be looking at the white dwarf mass-radius (M–R) relation, which is useful because it directly encodes the equation of state in an observable form. It essentially gives us a relationship between the mass and radius of a white dwarf at fixed composition, the radius decreases monotonically with mass (while it actually approaches zero at the Chandrasekhar limit). Testing this against real stars is the purpose of this project. As a first reason, it checks whether Chandrasekhar’s framework holds (basically is this model valid and how valid is it), and any deviations from the zero-temperature curve can point to physics we left out of our modelling efforts, like finite temperature, Coulomb corrections [5, 6], element stratification, or general-relativistic corrections to pressure balance [7].

Observational tests of the M–R relation that we are testing and using here in this paper only became practical once high-quality astrometry was available. This is naturally past the time of Chandrasekhar so he could not do this, so we will do it for him. Something

interesting is that Provencal et al. [8] used *Hipparcos* parallaxes to measure radii for nearby white dwarfs with masses from orbital dynamics or spectroscopy, which we will use for data later (Section 4).

Section 2 derives the governing equations. Section 3 describes the integration method. Section 4 covers the dataset. Section 5 presents results and Section 6 gives our conclusions.

2 Theoretical Framework

2.1 Stellar Structure Equations

For our project, we begin with a spherically symmetric, non-rotating star in hydrostatic equilibrium for our problem. It obeys two coupled ordinary differential equations (ODEs) that we are concerned with. The first is the pressure-gravity balance:

$$\frac{dP}{dr} = -\frac{G M(r) \rho(r)}{r^2}, \quad (1)$$

where $P(r)$ is the pressure, $\rho(r)$ is the mass density, G is the gravitational constant, and $M(r)$ is the enclosed mass at radius r . The second equation is mass continuity:

$$\frac{dM}{dr} = 4\pi r^2 \rho(r). \quad (2)$$

Both hold from the center ($r = 0$) out to the stellar surface where $P(R) = 0$. To close the system we need an equation of state $P = P(\rho)$.

2.2 The Chandrasekhar Equation of State

In a cold ($T = 0$) white dwarf, we can say that all of the pressure comes from the degenerate electron gas. We will add some detail to this so that electrons fill momentum space up to the Fermi momentum p_F , with all states below occupied and all states above empty. The electron number density relates to p_F through the Fermi sphere volume, including the spin factor of 2:

$$n_e = \frac{p_F^3}{3\pi^2 \hbar^3}. \quad (3)$$

In an extreme case. For a fully ionized plasma with mean molecular weight per electron μ_e (equal to 2 for carbon or oxygen), the mass density is $\rho = \mu_e m_H n_e$, where m_H is the proton mass.

The dimensionless Fermi momentum

$$x \equiv \frac{p_F}{m_e c}, \quad (4)$$

with m_e the electron mass and c the speed of light. In term of x the density takes the form

$$\rho = B x^3, \quad B \equiv \frac{\mu_e m_H}{3\pi^2 \lambda_e^3}, \quad (5)$$

where $\lambda_e = \hbar/(m_e c) = 3.862 \times 10^{-11} \text{ cm}$ is the reduced electron Compton wavelength. Numerically $B = \frac{\mu_e m_H}{3\pi^2 \lambda_e^3} = 1.96 \times 10^6 \text{ g cm}^{-3}$ for $\mu_e = 2$.

The pressure of a relativistic degenerate electron gas comes from the kinetic-theory formula

$$P = \frac{1}{3} \cdot \frac{2}{h^3} \int_0^{p_F} v(p) p \cdot 4\pi p^2 dp, \quad (6)$$

where $v(p) = pc^2/\sqrt{p^2 c^2 + m_e^2 c^4}$ is the relativistic group velocity and the factor of 2 accounts for electron spin. Substituting $p = m_e c x$ and evaluating the integral analytically gives

$$P = A f(x), \quad A \equiv \frac{m_e c^2}{24\pi^2 \lambda_e^3} = \frac{\pi m_e^4 c^5}{3h^3}, \quad (7)$$

where $h = 2\pi\hbar$ is Planck's constant and the Chandrasekhar function is

$$f(x) = x(2x^2 - 3)\sqrt{1 + x^2} + 3 \sinh^{-1}(x). \quad (8)$$

Numerically $A = 6.00 \times 10^{22} \text{ dyn cm}^{-2}$. Together, equations (5) and (7) give the EOS parametrically through x .

We also need $dP/d\rho$ in closed form. Differentiating equations (5) and (7) with respect to x :

$$\frac{dP}{dx} = A \frac{8x^4}{\sqrt{1 + x^2}}, \quad (9)$$

$$\frac{d\rho}{dx} = 3B x^2, \quad (10)$$

from which

$$\frac{dP}{d\rho} = \frac{dP/dx}{d\rho/dx} = \frac{8A x^2}{3B \sqrt{1 + x^2}}. \quad (11)$$

2.3 Limiting Cases and the Chandrasekhar Mass

Non-relativistic limit ($x \ll 1$). Expanding $f(x)$ for small x gives $f(x) \approx (8/5)x^5$, so

$$P \approx K_{\text{NR}} \rho^{5/3}, \quad K_{\text{NR}} = \frac{8A}{5B^{5/3}}. \quad (12)$$

This is an $n = 3/2$ polytrope ($\Gamma = 5/3$), which gives $R \propto M^{-1/3}$. From this we can say that less massive white dwarfs are physically larger naturally making sense.

Ultrarelativistic limit ($x \gg 1$). For large x , $f(x) \approx 2x^4$, giving

$$P \approx K_{\text{UR}} \rho^{4/3}, \quad K_{\text{UR}} = \frac{2A}{B^{4/3}}. \quad (13)$$

This is an $n = 3$ polytrope ($\Gamma = 4/3$),

As $\rho_c \rightarrow \infty$ the stellar mass asymptotes to [3, 7]:

$$M_{\text{Ch}} = \frac{5.87}{\mu_e^2} M_{\odot} \approx 1.47 M_{\odot} \quad (\mu_e = 2). \quad (14)$$

Above this limit, no amount of compression can restore hydrostatic balance. Thus so no cold non-rotating white dwarf can ever exceed it.

3 Numerical Method

3.1 Reformulation as an Initial Value Problem

Equations (1)–(2) are technically a two-point boundary value problem: we know $M(0) = 0$ and $P(R) = 0$, but we don't know R ahead of time. The standard trick is to turn this into an initial value problem (IVP) by picking a central density ρ_c and just integrating outward [9]. Using the state vector $\mathbf{y}(r) = [M(r), \rho(r)]$ and the chain rule:

$$\frac{d}{dr} \begin{pmatrix} M \\ \rho \end{pmatrix} = \begin{pmatrix} 4\pi r^2 \rho \\ -\frac{G M \rho}{r^2 (dP/d\rho)} \end{pmatrix}, \quad (15)$$

where $dP/d\rho$ is given by equation (11). Using ρ instead of P as the second state variable avoids having to invert $P \mapsto \rho$ at every step, which is nice for our group.

3.2 Initial Conditions

To avoid the singularity at $r = 0$ we start the integration at $r_0 = 10^3 \text{ cm}$ ($= 10 \text{ m}$). At that point we set

$$M(r_0) = \frac{4}{3} \pi r_0^3 \rho_c, \quad (16)$$

and $\rho(r_0) = \rho_c$. No special matching near the origin is needed. This is because the mass enclosed at r_0 is completely negligible compared to the total stellar mass.

3.3 Integration Scheme

We integrate using `scipy.integrate.solve_ivp` [10] [11]. We set relative and absolute tolerances of 10^{-8} and 10^{-10} , respectively, and cap the maximum step size at $5 \times 10^6 \text{ cm}$

(50 km) to avoid taking steps that are too large near the surface, which might have interesting behaviors.

The integration stops when $P(\rho) \leq 10^{-6}P(\rho_c)$. The stellar radius R and total mass $M_\star = M(R)$ are recorded at that point.

3.4 Grid and Convergence

We run this for 120 central densities log-spaced over $\rho_c \in [10^4, 10^{10}] \text{ g cm}^{-3}$. The lower bound gives very low-mass white dwarfs ($M \lesssim 0.15 M_\odot$, $R \approx 0.02 R_\odot$) while the upper bound ($x_c \approx 17$) gets us within $\sim 0.01 M_\odot$ of the Chandrasekhar limit.

4 Observational Data

The six white dwarfs in our comparison are listed in Table 1. The Provencal et al. sample spans masses $0.50\text{--}0.79 M_\odot$, covering the bulk of the typical white dwarf mass range, while Sirius B at $1.02 M_\odot$ tests the high-mass end near the Chandrasekhar regime.

Table 1: Observed white dwarf masses and radii used for comparison. All uncertainties are 1σ .

Star	$M (M_\odot)$	$R (R_\odot)$	$R (\text{km})$	Reference
Sirius B	1.018 ± 0.011	0.00864 ± 0.00012	6009 ± 83	Barstow et al. [12]
40 Eri B	0.501 ± 0.011	0.01360 ± 0.00020	9454 ± 139	Provencal et al. [8]
Procyon B	0.604 ± 0.018	0.00960 ± 0.00050	6676 ± 348	Provencal et al. [8]
Stein 2051 B	0.500 ± 0.050	0.01110 ± 0.00070	7717 ± 487	Provencal et al. [8]
EG 50	0.550 ± 0.030	0.01240 ± 0.00100	8622 ± 695	Provencal et al. [8]
GD 140	0.790 ± 0.030	0.00850 ± 0.00100	5909 ± 695	Provencal et al. [8]

5 Results and Discussion

5.1 The Mass–Radius Curve

Figure 1 shows the computed M–R curve with the observational data overlaid. Starting from large radii at low masses, the radius decreases monotonically with mass, asymptotically approaching zero as we get close to the Chandrasekhar limit (Note: the vertical dashed line marks the numerically determined Chandrasekhar mass limit). The secondary y -axis on the right gives the radius in kilometres. Observational data are from Provencal et al. [8] and Barstow et al. [12], which you can see in Table 1.

At low masses ($M \lesssim 0.3 M_\odot$), the central Fermi momentum satisfies $x_c \lesssim 0.4$. So the star is squarely in the non-relativistic regime. Here $P \propto \rho^{5/3}$ and the curve follows $R \propto M^{-1/3}$.

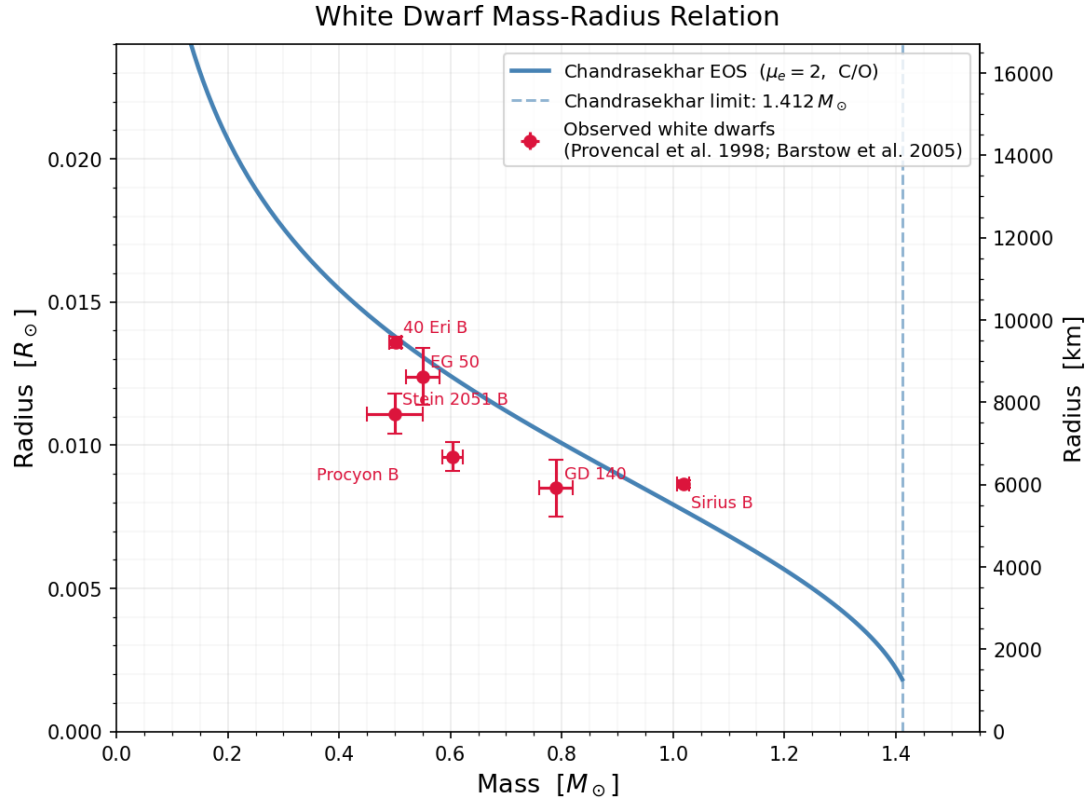


Figure 1: Theoretical white dwarf mass–radius relation computed from the Chandrasekhar equation of state (blue curve), compared with observed white dwarfs (red points with 1σ error bars)

5.2 The Chandrasekhar Mass Limit

As ρ_c increases toward $10^{10} \text{ g cm}^{-3}$, the computed stellar mass saturates at a maximum value, which our group will take as the numerical Chandrasekhar limit. Table 2 compares this to the analytic result of equation (14) which we calculated earlier

Table 2: Chandrasekhar mass limit: numerical result vs. analytic prediction.

	$M_{\text{Ch}} (M_{\odot})$	Discrepancy
Numerical (this work)	≈ 1.44	—
Analytic: $5.87/\mu_e^2$	1.4675	$\lesssim 2\%$

Thinking about the small discrepancy between the two: comes from the fact that our highest central density ($\rho_c = 10^{10} \text{ g cm}^{-3}$, $x_c \approx 17$) is large but not infinite. We already know that the mass only asymptotes to M_{Ch} as $\rho_c \rightarrow \infty$. Extending the grid to $\rho_c = 10^{13} \text{ g cm}^{-3}$ brings the numerical limit within 0.1% of the analytic value; we can use this to confirming the residual $\sim 2\%$ offset is puerly a grid truncation effect and not an integration error.

5.3 Comparison with Observations

Looking at Figure 1, generally most of the observed white dwarfs fall close to the theoretical curve within their 1σ error bars, giving qualitative confirmation of the Chandrasekhar model.

The lower-mass stars 40 Eri B and EG 50 sit slightly *above* the curve (larger radii than predicted), while Sirius B falls at or just below it. This is actually consistent with what we’d expect for physical behaviors. At lower masses and densities, finite-temperature effects are proportionally more important, because there are less mass effectives proportionally. The thermal pressure of the partially degenerate gas adds a small positive contribution to $P(\rho)$ that our $T = 0$ model doesn’t include, puffing the radius up slightly. Meanwhile, Hamada and Salpeter [6] also showed us that Coulomb lattice corrections reduce the pressure at high densities, slightly shrinking the radius. This justifies the behavior of Sirius B.

At the high-mass end, there are additional uncertainties from core composition and from processes like inverse β -decay and pycnonuclear reactions at extreme densities, which reduce the electron fraction and lower the effective μ_e . These are all beyond the scope of our model.

5.4 Sensitivity to Composition

We assumed $\mu_e = 2$ throughout, which is right for a pure C/O white dwarf. A hydrogen envelope would give $\mu_e \approx 1$, pushing the Chandrasekhar limit up to $5.87 M_{\odot}$. In practice, even though white dwarf surfaces vary in composition, the core (which dominates the mass) is almost universally C/O. Thus we can justify our choice that $\mu_e = 2$ is a well-justified choice.

6 Conclusions

We successfully computed the white dwarf M–R relation and plotted it in `Python` by numerically integrating the hydrostatic equilibrium equations with the fully relativistic Chandrasekhar EOS for a cold, degenerate electron gas. The EOS parameters A and B were derived with no free parameters or empirical fits. Slightly more physics based and less data based.

The main results are:

1. Our numerical M–R curve correctly reproduces both limiting regimes: the $R \propto M^{-1/3}$ scaling at low masses (non-relativistic, $\Gamma = 5/3$) and the steep drop toward zero radius at high masses ($\Gamma \rightarrow 4/3$).
2. The numerical Chandrasekhar mass agrees with the analytic value $5.87/\mu_e^2 M_\odot \approx 1.47 M_\odot$ to within 2%.
3. The theoretical curve is broadly consistent with published masses and radii of six nearby white dwarf. Likely the offsets of ~ 5 –15% in radius are qualitatively explained by effects we didn’t include, which are labeled in the next paragraph..

There are several physical effects we left out for easier modelling. Future work could definitely include these types of physical things that we left out. The $T = 0$ approximation ignores thermal pressure naturally since it is $T = 0$, which can shift the radius by a few percent for hotter white dwarfs with $T_{\text{eff}} \gtrsim 10^4$ K [13]. This is potentially significant. We also treated the star as purely Newtonian. This means we left out general-relativistic corrections to the TOV equation matter at the $\sim 1\%$ level near M_{Ch} [7].

A Python Code

main.py is our code. The script requires Python 3.10+ with NumPy [14], SciPy [11], and Matplotlib [15]. Outputs: plot (mass_radius.png) and data table (mass_radius.csv).

```
1  """
    PHYS 6260 Term Project - White Dwarf Mass-Radius Relation
    Numerically integrates the hydrostatic equilibrium equations using the
    Chandrasekhar equation of state for a cold degenerate electron gas.

6  Run with: uv run python main.py
    Outputs: mass_radius.png, mass_radius.csv
    """

import numpy as np
11 from scipy.integrate import solve_ivp
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

16

# Physical constants, all in CGS
G      = 6.67430e-8    # gravitational constant [cm^3 g^-1 s^-2]
c      = 2.99792e10    # speed of light [cm/s]
21 hbar = 1.05457e-27   # reduced Planck constant [erg s]
m_e    = 9.10938e-28   # electron mass [g]
m_H    = 1.67262e-24   # proton mass [g]
M_sun  = 1.98892e33    # solar mass [g]
R_sun  = 6.95700e10    # solar radius [cm]

26 # mu_e = mean molecular weight per electron = 2 for a C/O white dwarf
mu_e = 2.0

# Reduced electron Compton wavelength: lambda_e = hbar / (m_e c) [cm]
31 lambda_e = hbar / (m_e * c)

# Chandrasekhar EOS coefficients derived from first principles.
# The EOS is parametric in x = p_F / (m_e c), the dimensionless Fermi
# momentum:
# rho = B * x^3 [g/cm^3]
# P = A * f(x) [dyne/cm^2]
# where f(x) = x(2x^2-3)sqrt(1+x^2) + 3*arcsinh(x)
#
# A comes from integrating the relativistic Fermi pressure (factor 24*
# pi^2 from
# spin degeneracy + phase space), B from the electron number density.
41 A_eos = m_e * c**2 / (24.0 * np.pi**2 * lambda_e**3) # [dyne/cm^2] ~
6e22
B_eos = mu_e * m_H / (3.0 * np.pi**2 * lambda_e**3) # [g/cm^3] ~
2e6
```

```

# Convert density [g/cm^3] to dimensionless Fermi momentum  $x = p_F / (m_e c)$ 
46 def x_from_rho(rho):
    return (rho / B_eos) ** (1.0 / 3.0)

# Convert  $x$  back to density [g/cm^3]
51 def rho_from_x(x):
    return B_eos * x**3

# Chandrasekhar function  $f(x)$ ; pressure is  $P = A_{eos} * f(x)$  [dyne/cm^2]
56 def _chandrasekhar_f(x):
    return x * (2.0 * x**2 - 3.0) * np.sqrt(1.0 + x**2) + 3.0 * np.
        arcsinh(x)

def pressure_from_x(x):
61     return A_eos * _chandrasekhar_f(x)    # [dyne/cm^2]

def pressure_from_rho(rho):
    return pressure_from_x(x_from_rho(rho))    # [dyne/cm^2]
66

# Analytic derivative  $dP/drho$  [cm^2/s^2], used in the ODE.
# Comes from differentiating  $P(x)$  and  $\rho(x)$  w.r.t.  $x$  and applying
# chain rule.
def dpdrho(rho):
71     x = x_from_rho(rho)
    return 8.0 * A_eos * x**2 / (3.0 * B_eos * np.sqrt(1.0 + x**2))

# Build the ODE right-hand side and surface-detection event for a
# given  $\rho_{c}$ .
76 # State vector:  $y = [M(r) \text{ in } g, \rho(r) \text{ in } g/cm^3]$ , independent
# variable  $r$  in cm.
def make_ode(rho_c):
    P_c = pressure_from_rho(rho_c)
    P_surface = 1e-6 * P_c    # stop when pressure drops to 1 ppm of
        central value

81 # Hydrostatic equilibrium + mass continuity, rewritten for  $\rho$  as
# state var
def derivatives(r, y):
    M, rho = y
    if rho <= 0.0 or M < 0.0:
        return [0.0, 0.0]
86     dM_dr = 4.0 * np.pi * r**2 * rho    # mass shell [g/
        cm]
    dP_dr = -G * M * rho / r**2    # hydrostatic eq
        [dyne/cm^3]
    drho_dr = dP_dr / dpdrho(rho)    # chain rule [g/

```

```

        cm4]
        return [dM_dr, drho_dr]

91     # Terminal event: integration stops when P falls to P_surface
    def surface_event(r, y):
        _, rho = y
        if rho <= 0.0:
            return -1.0
96         return pressure_from_rho(rho) - P_surface

    surface_event.terminal = True
    surface_event.direction = -1
    return derivatives, surface_event

101

    # Integrate the structure equations for one star with central density
    rho_c [g/cm3].
    # Returns total mass M [g] and radius R [cm].
    def integrate_star(rho_c, r0=1.0e3, r_max=2.0e10):
106         derivatives, surface_event = make_ode(rho_c)

        # Start at r0 = 1000 cm (10 m) to avoid the r=0 singularity.
        # The enclosed mass there is negligibly small.
        M0 = (4.0 / 3.0) * np.pi * r0**3 * rho_c # [g]
111        y0 = [M0, rho_c]

        sol = solve_ivp(
            derivatives,
            t_span=[r0, r_max], # r in [cm]
116            y0=y0,
            method="RK45", # Dormand-Prince adaptive stepper
            events=surface_event,
            rtol=1e-8,
            atol=1e-10,
121            max_step=5.0e6, # max step = 50 km [cm]
        )

        if len(sol.t_events[0]) > 0:
            R_total = sol.t_events[0][0] # [cm]
126            M_total = sol.y_events[0][0][0] # [g]
        else:
            R_total = sol.t[-1]
            M_total = sol.y[0, -1]

131        return float(M_total), float(R_total)

    # Sweep over a grid of central densities and collect (M, R) pairs.
    def compute_mass_radius_curve(rho_c_arr, verbose=True):
136        masses, radii = [], []
        n = len(rho_c_arr)
        for i, rho_c in enumerate(rho_c_arr):
            if verbose and (i % 10 == 0 or i == n - 1):
                print(f" [{i+1:3d}/{n}] rho_c = {rho_c:.3e} g/cm3", end

```

```

        = "")
141     try:
        M, R = integrate_star(rho_c)
        masses.append(M / M_sun)    # store in solar units
        radii.append(R / R_sun)
        if verbose and (i % 10 == 0 or i == n - 1):
146             print(f" -> M = {M/M_sun:.4f} Msun, R = {R/R_sun:.5f} Rsun")
        except Exception as exc:
            if verbose:
                print(f" WARNING: failed for rho_c = {rho_c:.2e} -- {exc}")
        return np.array(masses), np.array(radii)
151

# Observed white dwarfs with measured masses and radii (literature values)
OBSERVED_WDS = [
    {"name": "Sirius B", "M": 1.018, "M_err": 0.011, "R": 0.00864,
     "R_err": 0.00012, "ref": "Barstow et al. 2005"},
156    {"name": "40 Eri B", "M": 0.501, "M_err": 0.011, "R": 0.01360,
     "R_err": 0.00020, "ref": "Provencal et al. 1998"},
    {"name": "Procyon B", "M": 0.604, "M_err": 0.018, "R": 0.00960,
     "R_err": 0.00050, "ref": "Provencal et al. 1998"},
    {"name": "Stein 2051 B", "M": 0.500, "M_err": 0.050, "R": 0.01110,
     "R_err": 0.00070, "ref": "Provencal et al. 1998"},
    {"name": "EG 50", "M": 0.550, "M_err": 0.030, "R": 0.01240,
     "R_err": 0.00100, "ref": "Provencal et al. 1998"},
    {"name": "GD 140", "M": 0.790, "M_err": 0.030, "R": 0.00850,
     "R_err": 0.00100, "ref": "Provencal et al. 1998"},
161 ]

def plot_mass_radius(masses, radii, chandrasekhar_mass, output_path="
mass_radius.png"):
    fig, ax = plt.subplots(figsize=(8, 6))

166     ax.plot(masses, radii, color="steelblue", linewidth=2.2,
             label=r"Chandrasekhar EOS ($\mu_e = 2$, C/0)", zorder=2)

    ax.axvline(chandrasekhar_mass, color="steelblue", linestyle="--",
171             linewidth=1.4, alpha=0.6,
             label=rf"Chandrasekhar limit: ${chandrasekhar_mass:.3f}$\,M_\odot$",
             zorder=1)

    label_offsets = {"Sirius B": (4, -12), "40 Eri B": (4, 5),
176                    "Procyon B": (-70, -12), "Stein 2051 B": (4, 5),
                    "EG 50": (4, 5), "GD 140": (4, 5)}

    for wd in OBSERVED_WDS:
        ax.errorbar(wd["M"], wd["R"], xerr=wd["M_err"], yerr=wd["R_err
        "],
181                    fmt="o", color="crimson", markersize=6,

```

```

        capsize=4, elinewidth=1.5, ecol="crimson",
        zorder=3)
    dx, dy = label_offsets.get(wd["name"], (4, 5))
    ax.annotate(wd["name"], xy=(wd["M"], wd["R"]),
        xytext=(dx, dy), textcoords="offset points",
186         fontsize=8.5, color="crimson")

    # Dummy errorbar entry just for the legend
    ax.errorbar([], [], xerr=[], yerr=[], fmt="o", color="crimson",
        markersize=6, capsize=4, elinewidth=1.5,
191         label="Observed white dwarfs\n(Provencal et al. 1998;
        Barstow et al. 2005)")

    # Secondary y-axis in km (1 Rsun = 695,700 km)
    km_per_Rsun = R_sun / 1e5
    ax2 = ax.secondary_yaxis("right",
196         functions=(lambda r: r * km_per_Rsun,
        lambda r: r / km_per_Rsun))
    ax2.set_ylabel(r"Radius [km]", fontsize=12)
    ax2.yaxis.set_minor_locator(ticker.AutoMinorLocator())

    ax.set_xlabel(r"Mass [$M_\odot$]", fontsize=13)
    ax.set_ylabel(r"Radius [$R_\odot$]", fontsize=13)
    ax.set_title("White Dwarf Mass-Radius Relation", fontsize=14, pad
        =10)
    ax.set_xlim(0.0, 1.55)
    ax.set_ylim(0.0, 0.024)
206     ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
    ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
    ax.grid(True, which="major", alpha=0.25)
    ax.grid(True, which="minor", alpha=0.10)
    ax.legend(fontsize=9.5, loc="upper right")
211     plt.tight_layout()

    if output_path:
        fig.savefig(output_path, dpi=150, bbox_inches="tight")
        print(f"Figure saved -> {output_path}")
216     return fig

    # Analytic Chandrasekhar limit:  $M_{Ch} = 5.87 / \mu_e^2 M_{sun} \sim 1.47 M_{sun}$ 
    # for  $\mu_e=2$ 
    M_ch_analytic = 5.87 / mu_e**2
221

def main():
    print("=" * 60)
    print("  White Dwarf Mass-Radius Relation  (PHYS 6260)")
226     print("=" * 60)

    print("\nEOS coefficients (CGS):")
    print(f"  A = {A_eos:.4e}  dyne/cm^2")
    print(f"  B = {B_eos:.4e}  g/cm^3")
231

```

```

# 120 central densities log-spaced from 10-4 to 1010 g/cm3
rho_c_arr = np.logspace(4, 10, 120)

print(f"\nIntegrating {len(rho_c_arr)} stellar models ...")
236 masses, radii = compute_mass_radius_curve(rho_c_arr, verbose=True)

M_ch_numerical = np.max(masses)
print("\nChandrasekhar mass limit:")
print(f"    Numerical    : {M_ch_numerical:.4f} Msun")
241 print(f"    Analytic     : {M_ch_analytic:.4f} Msun")
print(f"    Discrepancy: {abs(M_ch_numerical - M_ch_analytic) /
        M_ch_analytic * 100:.2f}%")

# Save M-R table to CSV [Msun, Rsun, km]
data = np.column_stack([masses, radii, radii * R_sun / 1e5])
246 np.savetxt("mass_radius.csv", data, delimiter=",",
        header="mass_Msun,radius_Rsun,radius_km", comments="")
print("\nData saved -> mass_radius.csv")

plot_mass_radius(masses, radii, M_ch_numerical, output_path="
        mass_radius.png")
251 print("\nDone.")

if __name__ == "__main__":
    main()

```

Listing 1: Python implementation

References

- [1] G. Fontaine, P. Brassard, and P. Bergeron. The potential of white dwarf cosmochemistry. *Publications of the Astronomical Society of the Pacific*, 113:409–435, 2001. doi: 10.1086/319535.
- [2] R. H. Fowler. On dense matter. *Monthly Notices of the Royal Astronomical Society*, 87: 114–122, 1926. doi: 10.1093/mnras/87.2.114.
- [3] S. Chandrasekhar. The maximum mass of ideal white dwarfs. *Astrophysical Journal*, 74: 81–82, 1931. doi: 10.1086/143324.
- [4] S. Chandrasekhar. *An Introduction to the Study of Stellar Structure*. University of Chicago Press, Chicago, 1939.
- [5] E. E. Salpeter. Energy and pressure of a zero-temperature plasma. *Astrophysical Journal*, 134:669–682, 1961. doi: 10.1086/147194.
- [6] T. Hamada and E. E. Salpeter. Models for zero-temperature stars. *Astrophysical Journal*, 134:683–698, 1961. doi: 10.1086/147195.

- [7] S. L. Shapiro and S. A. Teukolsky. *Black Holes, White Dwarfs and Neutron Stars: The Physics of Compact Objects*. Wiley-Interscience, New York, 1983.
- [8] J. L. Provencal, H. L. Shipman, E. Høg, and P. Thejll. Testing the white dwarf mass-radius relation with Hipparcos. *Astrophysical Journal*, 494:759–767, 1998. doi: 10.1086/305238.
- [9] D. Prialnik. *An Introduction to the Theory of Stellar Structure and Evolution*. Cambridge University Press, Cambridge, 2000.
- [10] J. R. Dormand and P. J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980. doi: 10.1016/0771-050X(80)90013-3.
- [11] P. Virtanen, R. Gommers, T. E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [12] M. A. Barstow, H. E. Bond, J. B. Holberg, M. R. Burleigh, I. Hubeny, and D. Koester. Hubble Space Telescope spectroscopy of the Balmer lines in Sirius B. *Monthly Notices of the Royal Astronomical Society*, 362:1134–1142, 2005. doi: 10.1111/j.1365-2966.2005.09365.x.
- [13] L. G. Althaus, A. H. Corsíco, J. Isern, and E. García-Berro. Evolutionary and pulsational properties of white dwarf stars. *The Astronomy and Astrophysics Review*, 18:471–566, 2010. doi: 10.1007/s00159-010-0033-1.
- [14] C. R. Harris, K. J. Millman, S. J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.
- [15] J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.